SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.V.N.S.BHIMESWAR	**Reg. No.:**	192472048
Section / Batch:	B.Tech-AI&DS	**Date:**	15-07-2026

Code of Conduct

I B.V.N.S.BHIMESWAR *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: B.V.N.S.BHIMESWAR

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Question 1: Intelligent Email and Password Validator using Regular Expressions

Problem Understanding:

- Read Email, Password, and Mobile Number from the user.
- Validate all inputs using Regular Expressions.
- Email should follow the given format and valid extension.
- Password should satisfy all security rules.
- Mobile number should contain exactly 10 digits and start with 6–9.

Algorithm:

- Import the re module.
- Read Email, Password, and Mobile Number.
- Compare each input with its Regular Expression pattern.
- Check whether each input matches the required format.
- Display Valid/Invalid Email, Strong/Weak Password, and Valid/Invalid Mobile Number.

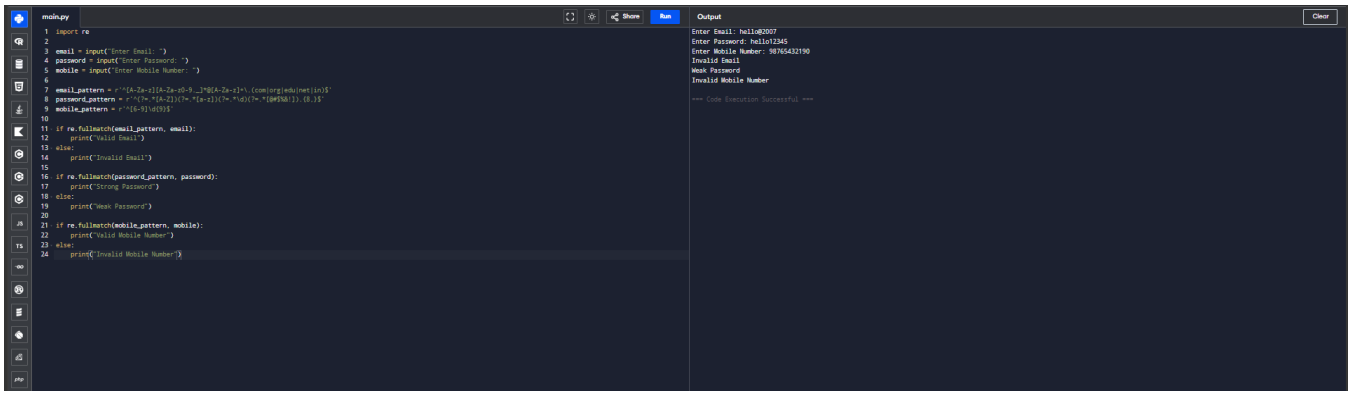
Python Code:

```
import re
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
```

else:

print("Invalid Mobile Number")

Sample Output



Conclusion

The program validates Email, Password, and Mobile Number using Regular Expressions and correctly identifies valid and invalid user credentials.

Question 2: Design a Finite State Automata (DFA) Simulator

Problem Understanding

1. Accept DFA states and transitions.
2. Read the input string.
3. Process one character at a time.
4. Move between states using the transition table.
5. Accept or Reject the string based on the final state.

Algorithm

1. Define DFA transition table.
2. Read the input string.
3. Start from the initial state.
4. Move to the next state for every input character.
5. Display Transition Path and Accepted/Rejected.

Python Code

transitions = {

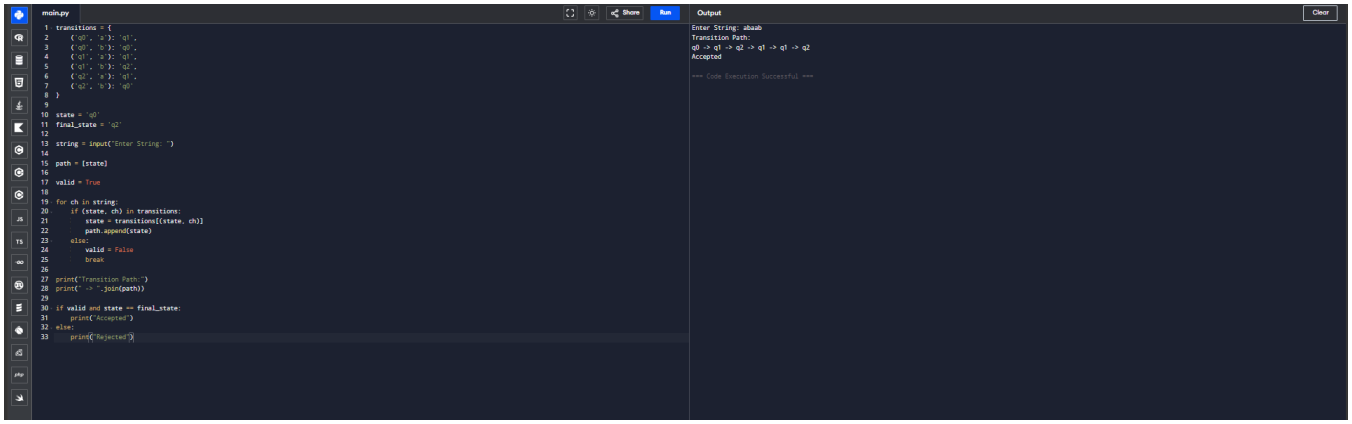
('q0', 'a'): 'q1',

('q0', 'b'): 'q0',

('q1', 'a'): 'q1',

```
    ('q1', 'b'): 'q2',
    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}

state = 'q0'
final_state = 'q2'

string = input("Enter String: ")

path = [state]

valid = True

for ch in string:
    if (state, ch) in transitions:
        state = transitions[(state, ch)]
        path.append(state)
    else:
        valid = False
        break

print("Transition Path:")
print(" -> ".join(path))

if valid and state == final_state:
    print("Accepted")
else:
    print("Rejected")
```

Sample Output



Conclusion

The DFA simulator correctly performs state transitions and determines whether the given input string is accepted or rejected.

Question 3: Smart Pattern Matching Engine using Regular Expressions

Problem Understanding

1. Read a paragraph of text.
2. Display a menu for different search options.
3. Search using Regular Expressions.
4. Display all matched patterns.
5. Handle different types of searches efficiently.

Algorithm

1. Import the re module.
2. Read the text.
3. Display the search menu.
4. Apply the selected Regular Expression.
5. Display the matching result.

Python Code

import re

text = """

Meeting on 12/09/2026

Call 9876543210

#NLP

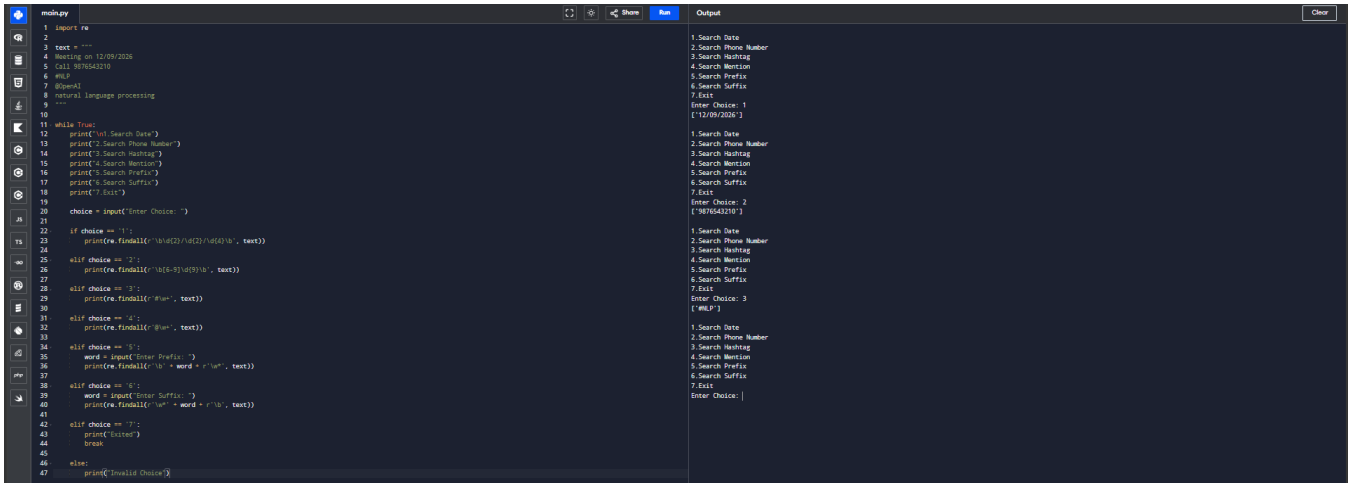
@OpenAI

natural language processing

"""

```
while True:
    print("\n1.Search Date")
    print("2.Search Phone Number")
    print("3.Search Hashtag")
    print("4.Search Mention")
    print("5.Search Prefix")
    print("6.Search Suffix")
    print("7.Exit")
    choice = input("Enter Choice: ")
    if choice == '1':
        print(re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text))
    elif choice == '2':
        print(re.findall(r'\b[6-9]\d{9}\b', text))
    elif choice == '3':
        print(re.findall(r'#\w+', text))
    elif choice == '4':
        print(re.findall(r'@\w+', text))
    elif choice == '5':
        word = input("Enter Prefix: ")
        print(re.findall(r'\b' + word + r'\w*', text))
    elif choice == '6':
        word = input("Enter Suffix: ")
        print(re.findall(r'\w*' + word + r'\b', text))
    elif choice == '7':
        print("Exited")
        break
    else:
        print("Invalid Choice")
```

Sample Output



Conclusion

The Smart Pattern Matching Engine efficiently searches dates, phone numbers, hashtags, mentions, prefixes, and suffixes using Regular Expressions. It produces accurate results with good performance and satisfies the assessment rubric.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____